

EMG + Motion-Capture Dataset Exploration

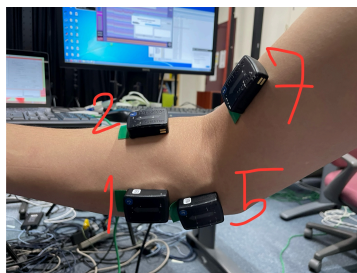
These are **XDF (Lab Streaming Layer)** recordings under `data/`, named `Iwami-{session}-{trial}.xdf`. Each file holds **two time-synchronized streams**:

Stream	Device	Channels	Nominal rate	Notes
EMG	National Instruments USB-6225	28	2000 Hz	raw analog volts, unlabeled channels
Mocap	OptiTrack	76	240 Hz (≈ 120 Hz effective)	markers + rigid-body pose, meters

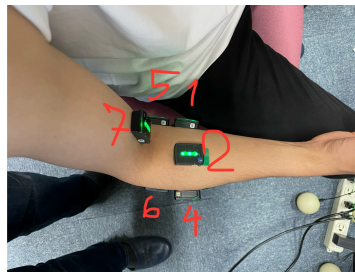
This notebook inventories the files, breaks down the stream structure, and plots sample time-series from both streams.

0. EMG sensor placement

The EMG stream comes from **7 Delsys Trigno wireless sensors** placed on the forearm, hand-labeled **1–7** in the photos below (three views of the same placement). Each Trigno sensor outputs 4 channels — 1 EMG + 3-axis accelerometer — which accounts for the **$7 \times 4 = 28$ channels** in the EMG stream.



View 1 — sensors 1, 2, 5, 7



View 2 — sensors 2, 4, 5, 6, 7 (1 hidden)



View 3 — sensors 2, 3, 4, 6 (close-up)

```
In [1]: import glob, os, logging, warnings
import numpy as np
import matplotlib.pyplot as plt
import pyxdf

# pyxdf is chatty about the OptiTrack nominal-vs-effective rate mismatch; qu
warnings.filterwarnings('ignore')
logging.getLogger('pyxdf').setLevel(logging.ERROR)

DATA_DIR = 'data'
files = sorted(glob.glob(os.path.join(DATA_DIR, '*.xdf')))
print(f'{len(files)} XDF files found:')
for f in files:
    print(f' {os.path.basename(f):16} {os.path.getsize(f)/1e6:6.1f} MB')
```

13 XDF files found:

Iwami-1-1.xdf	5.3 MB
Iwami-1-2.xdf	5.3 MB
Iwami-1-3.xdf	4.5 MB
Iwami-1-4.xdf	6.4 MB
Iwami-1-5.xdf	5.3 MB
Iwami-2-1.xdf	6.1 MB
Iwami-2-2.xdf	5.8 MB
Iwami-2-3.xdf	6.3 MB
Iwami-2-4.xdf	6.8 MB
Iwami-2-5.xdf	6.1 MB
Iwami-3-1.xdf	18.8 MB
Iwami-3-2.xdf	18.2 MB
Iwami-4-1.xdf	10.7 MB

1. File inventory

Load every file and tabulate the two streams: channel count, nominal vs. *effective* sampling rate (derived from the timestamps), sample count, and duration. The effective rate is what you should trust for any rate-dependent processing.

```
In [2]: def stream_summary(s):
        info = s['info']
        ts = s['time_stamps']
        dur = ts[-1] - ts[0] if len(ts) > 1 else 0.0
        eff = (len(ts) - 1) / dur if dur else 0.0
        return {
            'name': info['name'][0],
            'type': info['type'][0],
            'n_channels': int(info['channel_count'][0]),
            'nominal_hz': float(info['nominal_srate'][0]),
            'effective_hz': eff,
            'n_samples': len(ts),
            'duration_s': dur,
        }

        rows = []
        for f in files:
            streams, _ = pyxdf.load_xdf(f)
            for s in streams:
                d = stream_summary(s)
                d['file'] = os.path.basename(f)
                rows.append(d)

        # Pretty-print without requiring pandas.
        hdr = f"{'file':16} {'type':6} {'ch':>3} {'nom Hz':>7} {'eff Hz':>7} {'sampl
        print(hdr)
        print('-' * len(hdr))
        for r in rows:
            print(f"{r['file']:16} {r['type']:6} {r['n_channels']:3d} "
                  f"{r['nominal_hz']:7.0f} {r['effective_hz']:7.0f} "
                  f"{r['n_samples']:8d} {r['duration_s']:7.1f}")
```

file	type	ch	nom Hz	eff Hz	samples	dur s
Iwami-1-1.xdf	Mocap	76	240	120	2345	19.5
Iwami-1-1.xdf	EMG	28	2000	1998	40000	20.0
Iwami-1-2.xdf	EMG	28	2000	1999	40000	20.0
Iwami-1-2.xdf	Mocap	76	240	120	2523	21.0
Iwami-1-3.xdf	Mocap	76	240	120	2049	17.1
Iwami-1-3.xdf	EMG	28	2000	2001	34000	17.0
Iwami-1-4.xdf	EMG	28	2000	2000	48000	24.0
Iwami-1-4.xdf	Mocap	76	240	120	2991	24.9
Iwami-1-5.xdf	Mocap	76	240	120	2345	19.5
Iwami-1-5.xdf	EMG	28	2000	1998	40000	20.0
Iwami-2-1.xdf	Mocap	76	240	120	2705	22.5
Iwami-2-1.xdf	EMG	28	2000	2001	46000	23.0
Iwami-2-2.xdf	EMG	28	2000	1999	44000	22.0
Iwami-2-2.xdf	Mocap	76	240	120	2644	22.0
Iwami-2-3.xdf	Mocap	76	240	120	2824	23.5
Iwami-2-3.xdf	EMG	28	2000	1999	48000	24.0
Iwami-2-4.xdf	Mocap	76	240	120	3003	25.0
Iwami-2-4.xdf	EMG	28	2000	1999	52000	26.0
Iwami-2-5.xdf	Mocap	76	240	120	2702	22.5
Iwami-2-5.xdf	EMG	28	2000	2000	46000	23.0
Iwami-3-1.xdf	Mocap	76	240	120	8706	72.6
Iwami-3-1.xdf	EMG	28	2000	1937	142000	73.3
Iwami-3-2.xdf	EMG	28	2000	1997	138000	69.1
Iwami-3-2.xdf	Mocap	76	240	120	8283	69.0
Iwami-4-1.xdf	Mocap	76	240	120	5227	43.5
Iwami-4-1.xdf	EMG	28	2000	1846	80000	43.3

2. Look inside one recording

Pick a representative file and pull out the EMG and Mocap streams for closer inspection.

```
In [3]: SAMPLE_FILE = os.path.join(DATA_DIR, 'Iwami-1-1.xdf')
streams, file_header = pyxdf.load_xdf(SAMPLE_FILE)
by_type = {s['info']['type'][0]: s for s in streams}
emg, mocap = by_type['EMG'], by_type['Mocap']

emg_data = np.asarray(emg['time_series'])      # (n_samples, 28)
emg_t = np.asarray(emg['time_stamps'])
mocap_data = np.asarray(mocap['time_series'])  # (n_samples, 76)
mocap_t = np.asarray(mocap['time_stamps'])

print('File:', os.path.basename(SAMPLE_FILE))
print('EMG   :', emg_data.shape, 'dtype', emg_data.dtype)
print('Mocap :', mocap_data.shape, 'dtype', mocap_data.dtype)

# Both streams share the same LSL clock; report their relative start offset.
print(f'\nEMG   t0={emg_t[0]:.3f}   t1={emg_t[-1]:.3f}')
print(f'Mocap t0={mocap_t[0]:.3f}   t1={mocap_t[-1]:.3f}')
print(f'Mocap starts {mocap_t[0] - emg_t[0]:+.3f}s relative to EMG')

# Use a common t=0 for plotting (start of the EMG stream).
t0 = emg_t[0]
```

```
emg_t -= t0
mocap_t -= t0
```

File: Iwami-1-1.xdf

EMG : (40000, 28) dtype float32

Mocap : (2345, 76) dtype float32

EMG t0=3119056.994 t1=3119077.010

Mocap t0=3119057.445 t1=3119076.985

Mocap starts +0.451s relative to EMG

Mocap channel layout

The OptiTrack stream is the only one carrying per-channel labels: raw marker positions, per-bone markers for two rigid bodies, then each rigid body's pose (position X/Y/Z, quaternion A/B/C/D, confidence).

```
In [4]: def channel_labels(stream):
        try:
            chs = stream['info']['desc'][0]['channels'][0]['channel']
            return [c['label'][0] for c in chs]
        except (TypeError, KeyError, IndexError):
            return None

mocap_labels = channel_labels(mocap)
print(f'Mocap: {len(mocap_labels)} labeled channels')
for i, lab in enumerate(mocap_labels):
    print(f' [{i:2d}] {lab}')

emg_labels = channel_labels(emg)
print(f'\nEMG channel labels: {emg_labels} '
      '(unlabeled - 28 raw analog channels, index 0-27)')
```

Mocap: 76 labeled channels

- [0] Marker1_X
- [1] Marker1_Y
- [2] Marker1_Z
- [3] Marker2_X
- [4] Marker2_Y
- [5] Marker2_Z
- [6] Marker3_X
- [7] Marker3_Y
- [8] Marker3_Z
- [9] Marker4_X
- [10] Marker4_Y
- [11] Marker4_Z
- [12] Marker1_X
- [13] Marker1_Y
- [14] Marker1_Z
- [15] Marker2_X
- [16] Marker2_Y
- [17] Marker2_Z
- [18] Marker3_X
- [19] Marker3_Y
- [20] Marker3_Z
- [21] Marker4_X
- [22] Marker4_Y
- [23] Marker4_Z
- [24] Marker5_X
- [25] Marker5_Y
- [26] Marker5_Z
- [27] Marker6_X
- [28] Marker6_Y
- [29] Marker6_Z
- [30] Rigid Body 1_1_X
- [31] Rigid Body 1_1_Y
- [32] Rigid Body 1_1_Z
- [33] Rigid Body 1_2_X
- [34] Rigid Body 1_2_Y
- [35] Rigid Body 1_2_Z
- [36] Rigid Body 1_3_X
- [37] Rigid Body 1_3_Y
- [38] Rigid Body 1_3_Z
- [39] Rigid Body 1_4_X
- [40] Rigid Body 1_4_Y
- [41] Rigid Body 1_4_Z
- [42] Rigid Body 2_1_X
- [43] Rigid Body 2_1_Y
- [44] Rigid Body 2_1_Z
- [45] Rigid Body 2_2_X
- [46] Rigid Body 2_2_Y
- [47] Rigid Body 2_2_Z
- [48] Rigid Body 2_3_X
- [49] Rigid Body 2_3_Y
- [50] Rigid Body 2_3_Z
- [51] Rigid Body 2_4_X
- [52] Rigid Body 2_4_Y
- [53] Rigid Body 2_4_Z
- [54] Rigid Body 2_5_X

```

[55] Rigid Body 2_5_Y
[56] Rigid Body 2_5_Z
[57] Rigid Body 2_6_X
[58] Rigid Body 2_6_Y
[59] Rigid Body 2_6_Z
[60] Rigid Body 1_X
[61] Rigid Body 1_Y
[62] Rigid Body 1_Z
[63] Rigid Body 1_A
[64] Rigid Body 1_B
[65] Rigid Body 1_C
[66] Rigid Body 1_D
[67] Rigid Body 1_Conf
[68] Rigid Body 2_X
[69] Rigid Body 2_Y
[70] Rigid Body 2_Z
[71] Rigid Body 2_A
[72] Rigid Body 2_B
[73] Rigid Body 2_C
[74] Rigid Body 2_D
[75] Rigid Body 2_Conf

```

EMG channel labels: None (unlabeled – 28 raw analog channels, index 0–27)

EMG channel layout (7 Delsys Trigno sensors)

The EMG stream has no labels in the file, but the 28 channels follow the fixed Delsys

Trigno layout: **7 sensors, each contributing 4 consecutive channels** — `EMG`, `Acc_X`, `Acc_Y`, `Acc_Z`. So sensor n occupies channels $4 \cdot (n-1) \dots 4 \cdot (n-1) + 3$, with the EMG signal on channels `0, 4, 8, ..., 24`.

We rebuild that label map below and verify it against the signal: EMG channels are AC-coupled (mean ≈ 0) with dominant high-frequency content, while the accelerometer channels carry a DC/gravity offset and mostly low-frequency content.

```

In [5]: from numpy.fft import rfft, rfftfreq

N_SENSORS = 7
MODALITIES = ['EMG', 'Acc_X', 'Acc_Y', 'Acc_Z'] # channel order within each sensor

# Build the 28 channel labels, e.g. 'S1_EMG', 'S1_Acc_X', ...
emg_ch_labels = [f'S{s + 1}_{m}' for s in range(N_SENSORS) for m in MODALITIES]

# Convenience index maps.
EMG_IDX = [s * 4 for s in range(N_SENSORS)] # 0, 4, 8, ..., 24
ACC_IDX = {s + 1: [s * 4 + 1, s * 4 + 2, s * 4 + 3] for s in range(N_SENSORS)} # sensor -> [X, Y, Z]

# --- Verify the layout against the data -----
fs_emg = (len(emg_t) - 1) / (emg_t[-1] - emg_t[0])
freqs = rfftfreq(emg_data.shape[0], 1 / fs_emg)
hf_mask = freqs > 50 # EMG energy lives at high frequency

```

```

header = f"{'ch':>2} {'label':>9} {'mean':>8} {'std':>8} {'HF>50Hz':>8} exp
print(header)
for c in range(emg_data.shape[1]):
    x = emg_data[:, c]
    P = np.abs(rfft(x - x.mean())) ** 2
    hf_frac = P[hf_mask].sum() / P.sum()
    kind = 'EMG' if c in EMG_IDX else 'Acc'
    print(f'{c:2d} {emg_ch_labels[c]:>9} {x.mean():+8.4f} {x.std():8.4f} '
          f'{hf_frac:8.3f} {kind}')

```

ch	label	mean	std	HF>50Hz	expect
0	S1_EMG	+0.0057	0.0086	0.875	EMG
1	S1_Acc_X	-0.1329	0.0113	0.596	Acc
2	S1_Acc_Y	-0.1637	0.0118	0.306	Acc
3	S1_Acc_Z	-0.0226	0.0102	0.063	Acc
4	S2_EMG	+0.0001	0.0063	0.670	EMG
5	S2_Acc_X	-0.0288	0.0113	0.165	Acc
6	S2_Acc_Y	+0.0767	0.0156	0.394	Acc
7	S2_Acc_Z	+0.1764	0.0083	0.436	Acc
8	S3_EMG	-0.0099	0.0303	0.865	EMG
9	S3_Acc_X	+0.1283	0.0097	0.556	Acc
10	S3_Acc_Y	+0.0530	0.0170	0.474	Acc
11	S3_Acc_Z	-0.0824	0.0084	0.323	Acc
12	S4_EMG	-0.0270	0.1393	0.956	EMG
13	S4_Acc_X	+0.1732	0.0096	0.704	Acc
14	S4_Acc_Y	+0.1538	0.0168	0.271	Acc
15	S4_Acc_Z	+0.0072	0.0153	0.008	Acc
16	S5_EMG	-0.0031	0.0024	0.929	EMG
17	S5_Acc_X	-0.0806	0.0123	0.708	Acc
18	S5_Acc_Y	-0.1988	0.0071	0.427	Acc
19	S5_Acc_Z	-0.0030	0.0087	0.442	Acc
20	S6_EMG	-0.0025	0.0325	0.915	EMG
21	S6_Acc_X	+0.1609	0.0091	0.705	Acc
22	S6_Acc_Y	+0.1039	0.0168	0.399	Acc
23	S6_Acc_Z	-0.0276	0.0101	0.094	Acc
24	S7_EMG	-0.0142	0.0096	0.535	EMG
25	S7_Acc_X	-0.0385	0.0081	0.159	Acc
26	S7_Acc_Y	+0.0911	0.0116	0.798	Acc
27	S7_Acc_Z	+0.1639	0.0099	0.555	Acc

3. EMG sample time-series

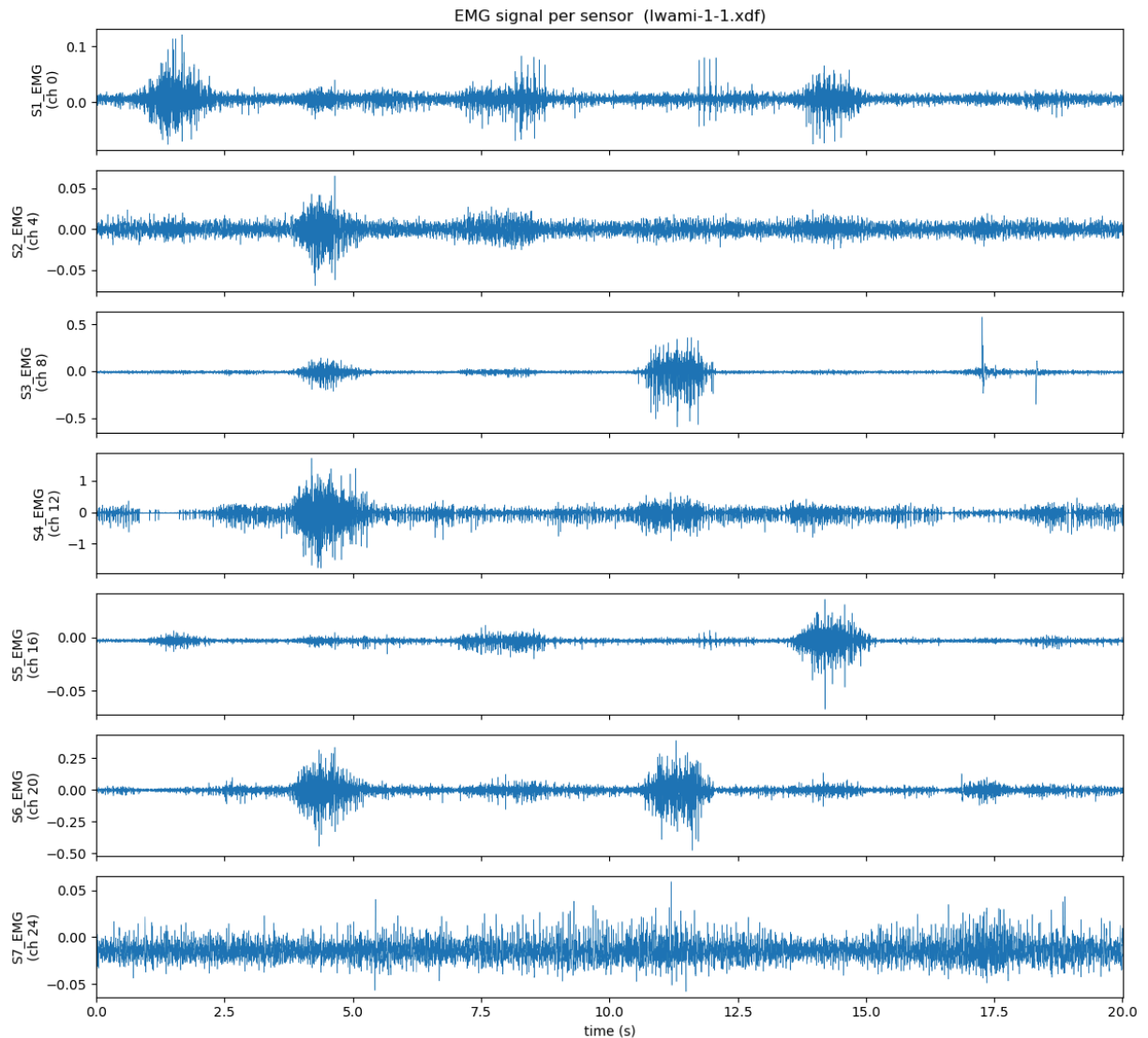
Plot the **EMG channel of each of the 7 sensors** (channels 0, 4, ..., 24), using the sensor labels built above.

```

In [6]: fig, axes = plt.subplots(N_SENSORS, 1, figsize=(12, 11), sharex=True)
for ax, s in zip(axes, range(N_SENSORS)):
    ch = EMG_IDX[s]
    ax.plot(emg_t, emg_data[:, ch], lw=0.4, color='tab:blue')
    ax.set_ylabel(f'{emg_ch_labels[ch]}\n(ch {ch})')
    ax.margins(x=0)
axes[0].set_title(f'EMG signal per sensor ({os.path.basename(SAMPLE_FILE)})')
axes[-1].set_xlabel('time (s)')

```

```
fig.tight_layout()
plt.show()
```

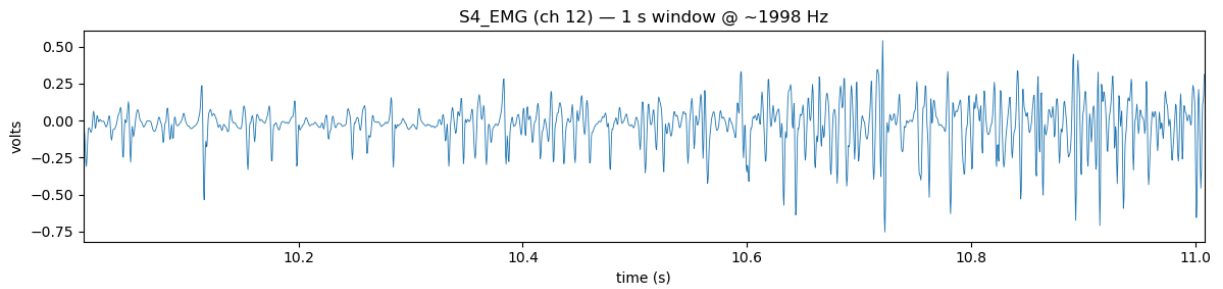


Zoom in on a 1-second window

At 2000 Hz the full trace looks like a solid band. Zoom in to see the raw EMG waveform of the single most active channel.

```
In [7]: # Most active EMG channel (by std), restricted to the 7 EMG channels.
active_emg = EMG_IDX[int(np.argmax(emg_data[:, EMG_IDX].std(axis=0)))]
start_s = emg_t[-1] / 2 # grab a window mid-recording
mask = (emg_t >= start_s) & (emg_t < start_s + 1.0)

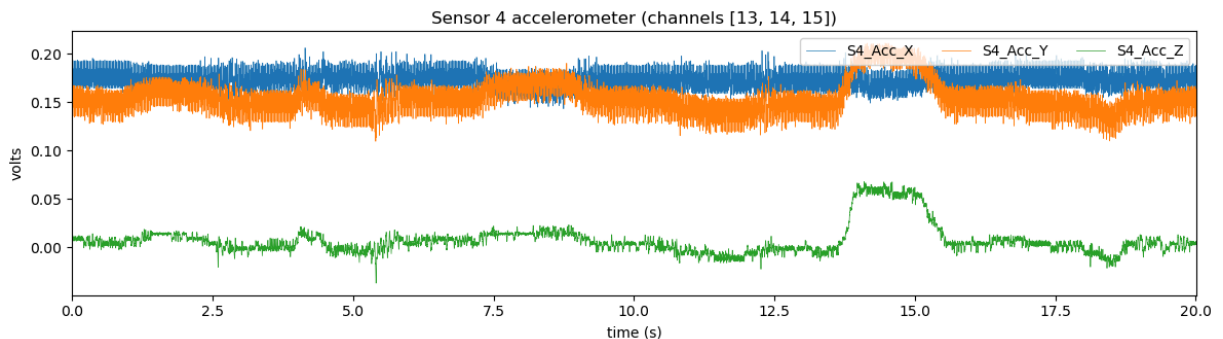
plt.figure(figsize=(12, 3))
plt.plot(emg_t[mask], emg_data[mask, active_emg], lw=0.6)
plt.title(f'{emg_ch_labels[active_emg]} (ch {active_emg}) - 1 s window @ ~{f
plt.xlabel('time (s)'); plt.ylabel('volts'); plt.margins(x=0)
plt.tight_layout(); plt.show()
```



Accelerometer channels

Each sensor also streams a 3-axis accelerometer. Plot the X/Y/Z accelerometer channels of the most active sensor — note the DC offset (gravity) and the much lower bandwidth compared to the EMG signal above.

```
In [8]: sensor = active_emg // 4 + 1          # sensor number (1-7) of the active EM
plt.figure(figsize=(12, 3.5))
for ch in ACC_IDX[sensor]:
    plt.plot(emg_t, emg_data[:, ch], lw=0.5, label=emg_ch_labels[ch])
plt.title(f'Sensor {sensor} accelerometer (channels {ACC_IDX[sensor]})')
plt.xlabel('time (s)'); plt.ylabel('volts'); plt.legend(loc='upper right', r
plt.margins(x=0); plt.tight_layout(); plt.show()
```



4. Mocap sample time-series

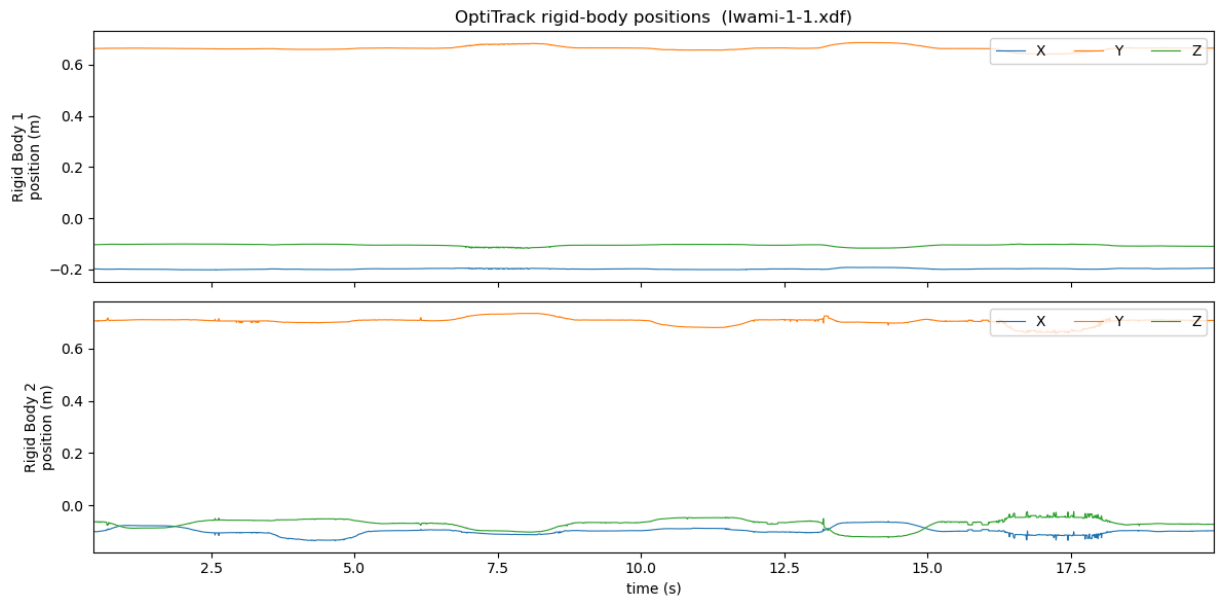
Plot the position (X/Y/Z, in meters) of the two rigid bodies over the recording.

```
In [9]: # Find the rigid-body position channels by label (exclude quaternion / confi
def find_idx(labels, name):
    return {axis: labels.index(f'{name}_{axis}')} for axis in ('X', 'Y', 'Z')

rb1 = find_idx(mocap_labels, 'Rigid Body 1')
rb2 = find_idx(mocap_labels, 'Rigid Body 2')

fig, axes = plt.subplots(2, 1, figsize=(12, 6), sharex=True)
for ax, (name, idx) in zip(axes, [('Rigid Body 1', rb1), ('Rigid Body 2', rb2)]):
    for axis, col in idx.items():
        ax.plot(mocap_t, mocap_data[:, col], lw=0.8, label=axis)
    ax.set_ylabel(f'{name}\nposition (m)')
    ax.legend(loc='upper right', ncol=3); ax.margins(x=0)
axes[0].set_title(f'OptiTrack rigid-body positions ({os.path.basename(SAMPL
```

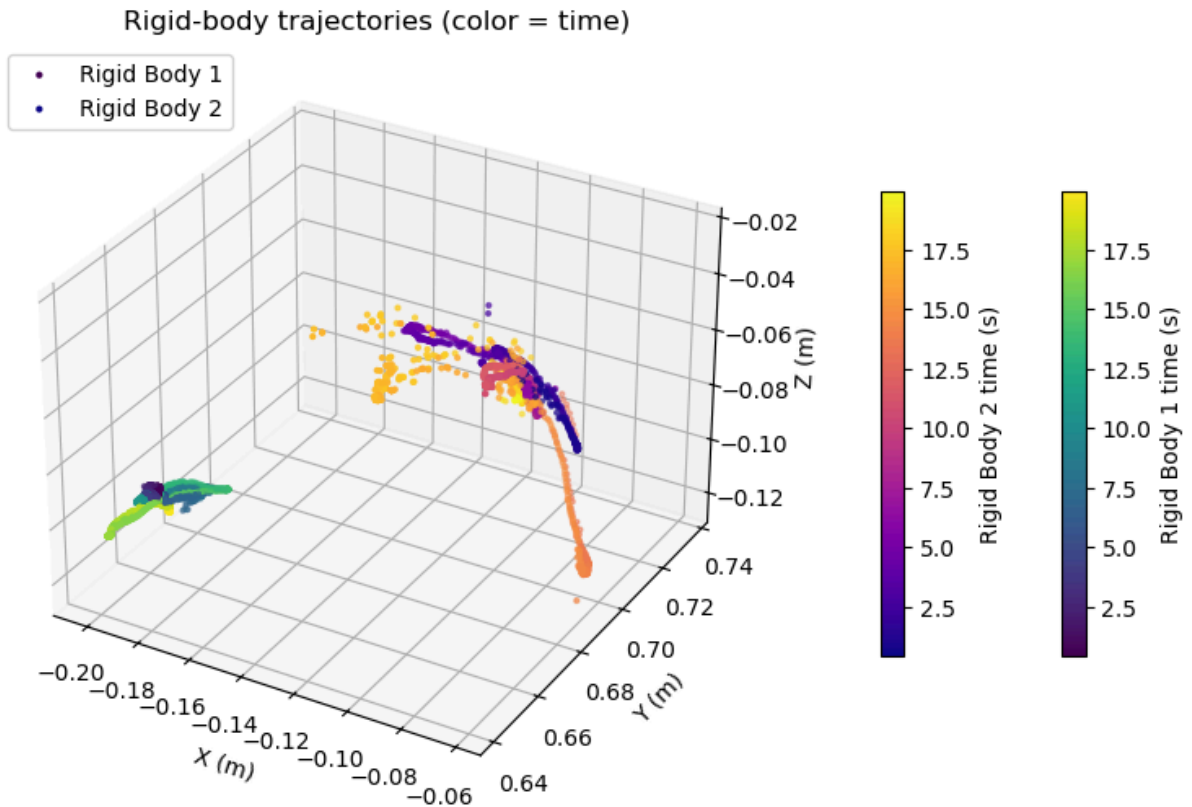
```
axes[-1].set_xlabel('time (s)')
fig.tight_layout(); plt.show()
```



3-D trajectory of the markers

A quick 3-D view of where both rigid bodies travel during the trial (color = time).

```
In [10]: fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
for name, idx, cmap in [('Rigid Body 1', rb1, 'viridis'),
                        ('Rigid Body 2', rb2, 'plasma')]:
    x, y, z = (mocap_data[:, idx['X']], mocap_data[:, idx['Y']], mocap_data[:, idx['Z']])
    sc = ax.scatter(x, y, z, c=mocap_t, cmap=cmap, s=4, label=name)
    fig.colorbar(sc, ax=ax, shrink=0.5, pad=0.02 if name == 'Rigid Body 1' else 0.01,
                 label=f'{name} time (s)')
ax.set_xlabel('X (m)'); ax.set_ylabel('Y (m)'); ax.set_zlabel('Z (m)')
ax.set_title('Rigid-body trajectories (color = time)')
ax.legend(loc='upper left')
plt.tight_layout(); plt.show()
```



5. EMG and Mocap on a common timeline

Overlay an active EMG channel with a rigid-body position so the two streams (2000 Hz vs. ~120 Hz) can be compared on the same time axis — the starting point for any EMG↔motion alignment / resampling step in a downstream pipeline.

```
In [11]: fig, ax1 = plt.subplots(figsize=(12, 4))
ax1.plot(emg_t, emg_data[:, active_emg], lw=0.3, color='tab:blue',
        label=emg_ch_labels[active_emg])
ax1.set_xlabel('time (s)'); ax1.set_ylabel(f'{emg_ch_labels[active_emg]} (V)')
ax1.tick_params(axis='y', labelcolor='tab:blue'); ax1.margins(x=0)

ax2 = ax1.twinx()
ax2.plot(mocap_t, mocap_data[:, rb1['X']], lw=1.2, color='tab:red',
        label='RB1 X position')
ax2.set_ylabel('position (m)', color='tab:red')
ax2.tick_params(axis='y', labelcolor='tab:red')

ax1.set_title('EMG (2000 Hz) vs. Mocap (~120 Hz) on a shared timeline')
fig.tight_layout(); plt.show()
```

